Assembly language instructions:

Transfer instructions

mov d,s

PUSH s

POP d

xCHG d,s

xLAT

CMOVcc d,s

PUSH s / POP d

push eax ⇔ (mov [esp], eax ; sub esp, 4)

pop eax (=) ~~add~~

mov eax, [esp]

add esp, 4

; EAX = 11223344h

XCHG 01, 02 ↪ mov ECX, EAX ; mov EAX, EBX ; mov EBX, ECX

XCHG EAX, EBX

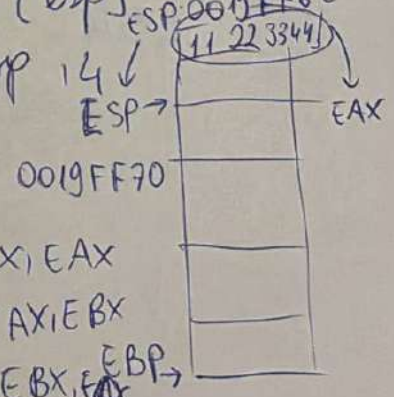
pushad / pusha ; EAX ... EDI

popad / popa ; pop EDI, ..., EAX

PUSHF; 4 EFLAGS → stack

POPF; stack → EFLAGS

SETcc d; d ← 1 otherwise 0



XLAT "translate" the byte from AL to another byte

; translate a digit from base 10 → base 16 ASCII

```
data
tabel_1 db '012....9ABCDEF';
mr resb 1                    [reg_reg] XLAT
:
code
:
    mov ebx, tabel_1 ; ← EBX offset of tabel_1

    mov al,3
    XLAT; AL ← DS:[EBX+AL]
    ES XLAT ; AL ← ES:[EBX+AL]
```

LEA reg,0 ; Load Effective Address

lea eax,[v] ; EAX offset of v

↔ mov eax,v

0 - operand of LEA may becam addressing expression

lea [ebx+v-6] ↔ mov eax, [ebx+v-6]

```
mov eax, [number]
lea eax, [eax * 2] ; EAX ← number * 2
lea eax, [eax*4 + eax] ; EAX ← nb * 5
                         (eax * 4 + eax
```

flag instructions

LAHF - Load into AH (from Flags)
SF, ZF, AF, PF, CF →
bits 7, 6, 4, 2 and 0 from AH

SAHF → bits 7, 6, 4, 2 and 0 from AH → SF, ZF, ...

type conversions

- CBW ; AX ← AL
- CWD ; DX.AX ← AX
- CWDE
- CDQ

MOVZX
MOVSX

1100

```
mov ah, 0C8h
movsx ebx, ah
; EBX = FFFFFFC8h
movzx ebx, ah
; EBX = 000000C8h
```

short jump vs long jump ?

- 4 characters will be arranged with ...

```
label_1:
    mov eax,1
    :
    jmp label2 - far jump
    resd 1000h
label_2:
    mov ebx,17
    :
    :
    loop label_1 , distance between loop and label_1> 127 bytes
```

dec ecx
~~jmp label_1~~ ?! { dec ECX
jnz label_1

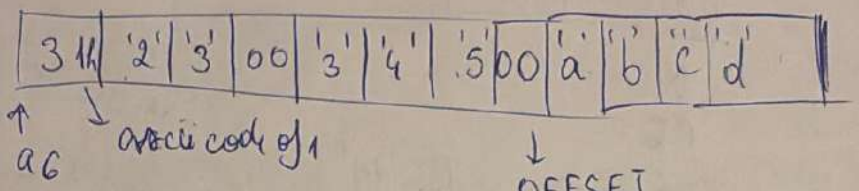
ECX=2

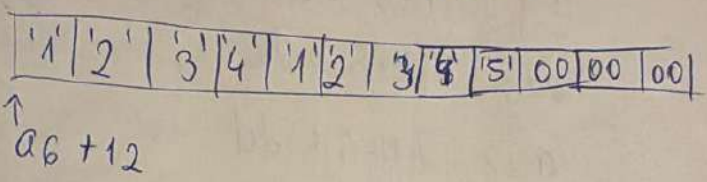
```
label_1: <-
    mov eax,1
    :
    jmp label_2
    resd 1000h
label_3:
    jmp label 1
label 2:
    mov ebx,17
    loop label_3
```

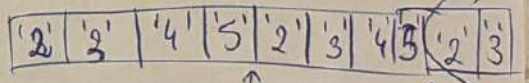
will be organized with more than 1 byte with little endian in mind

Memory layout : string constants

```
a6 dd '123', '345', 'abcd'
   dd '1234'
   dd '12345'
```

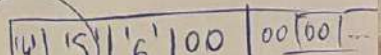







```
a7_1 dw '23', '45'
a7_2 dw '2345'
a7_3 dw '23456'
```

a7_1 a7_2 a7_3

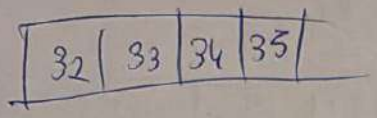



```
mov eax, [a7_3] ; EAX = '5432' = 35 34 33 32

mov ah, [a7_3] ; AH = '2' = 32h

mov ax, [a7_3] ; AX = '32' = 33 32h
                              61h = 'a'

mov eax, 'abcd' ; EAX = 0x64636261
```

- a character constant with more than 1 byte will be arranged with little endian in mind

mov dword [a6], '2345' ; mov dword ptr DS:[401000], 35343332

olly DBG



a6

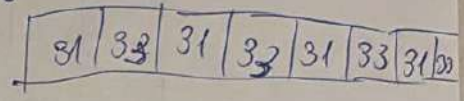
a8 TIMES 4 db '13'

a9 TIMES 4 dw '13'

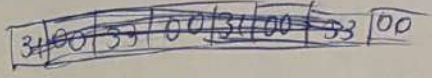
a10 TIMES 4 dw '1', '3'

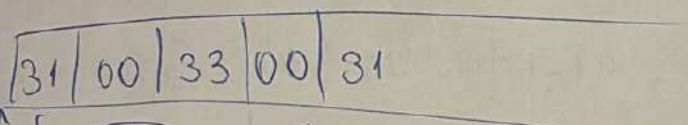
a11 TIMES 4 dd '13'

a8



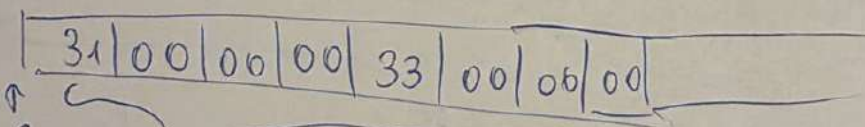
-11 -11-





a10

x4



a11 x4

Location counter and pointer arithmetic

data

a db 1,2,3,4; 01 02 03 04

lg db $-a; 4

lg-1 db $-data; syntax error

fg-3 dw data-a ; linking error

db a-$; -5 = 0FBh 00000101

c equ a-$; -6 = 0FAh 11110 14

e db a-$; e ← (-6)10

x dw x ✓ ; x=0710h least sign word and store it starting from offset x

x db x ✗ syntax error

16 bits

32 bits not 8! word can be written like that

label_1 equ label_1 ; label_1 = 0!!

Nasm bug!!